OpenID Connect For .NET Developers

nestenius.se

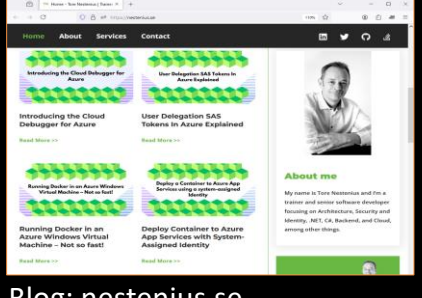
MVP Microsoft Most Valuable Professional

https://nestenius.se

1

About Tore Nestenius

Work: tn-data.se



Blog: nestenius.se

meetup.com/net-skane

Have a question about this session or beyond?

tore@tn-data.se

https://nestenius.se

2

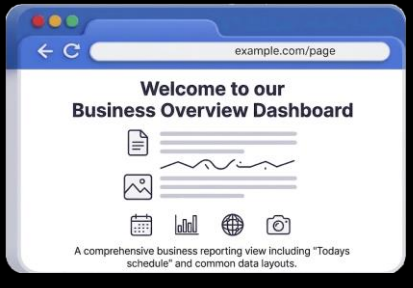
First, One Important Thing

Do Ask Questions!

The New Project

The New Project

You Just Built This Awesome ASP.NET Core Web App



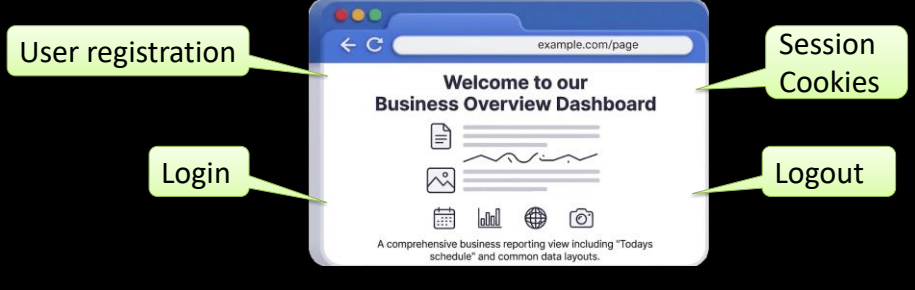
What Did We Forget?

https://nestenius.se

5

The New Project

Your Client Asks You To Add Authentication!



But It Gets Worse...

https://nestenius.se

6

The New Project

The Client Has More Requirements

So, How Do We Support This?

7

We Need To Use OpenID Connect!

OpenID®

But How Many of Us Can Explain It?

https://nestenius.se

8

OpenID Connect

Example: I Have My Photos in Google Photos

You

Google Photos

I Want a Photo Printing Service to Print Them

OpenID Connect

I Need to Give the Photo Printer Access to My Photos

You

Photo Printer

Google Photos

How Do I Give It Access?

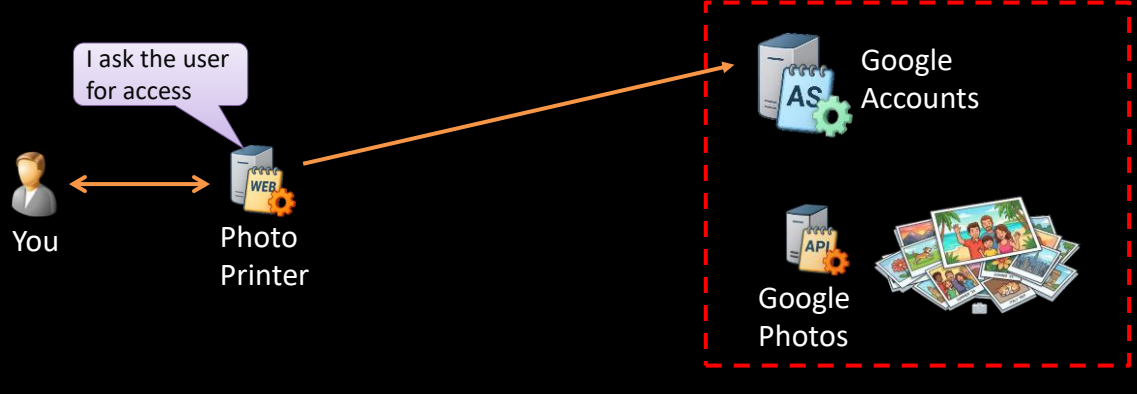
The Photo Printer Needs an Access Token

How Does the Photo Printer Get the Token?

https://nestenius.se

11

It Asks Google Accounts for a Token



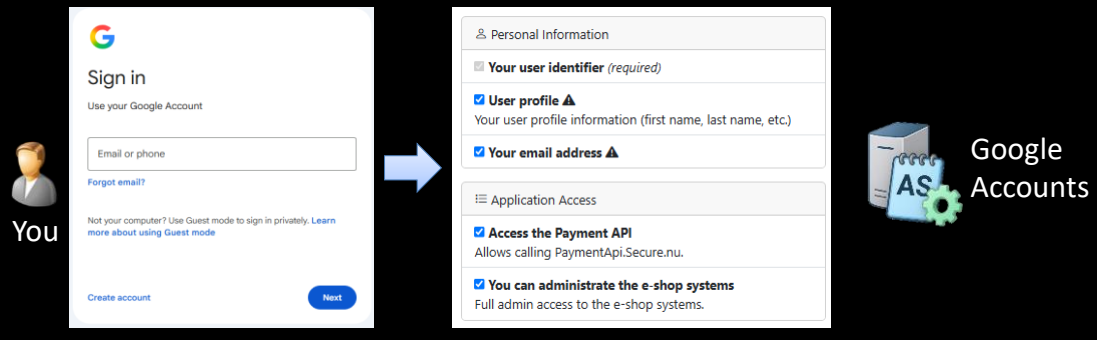
What Happens Next?

https://nestenius.se

12

OpenID Connect

The User Authenticate and Give Consent



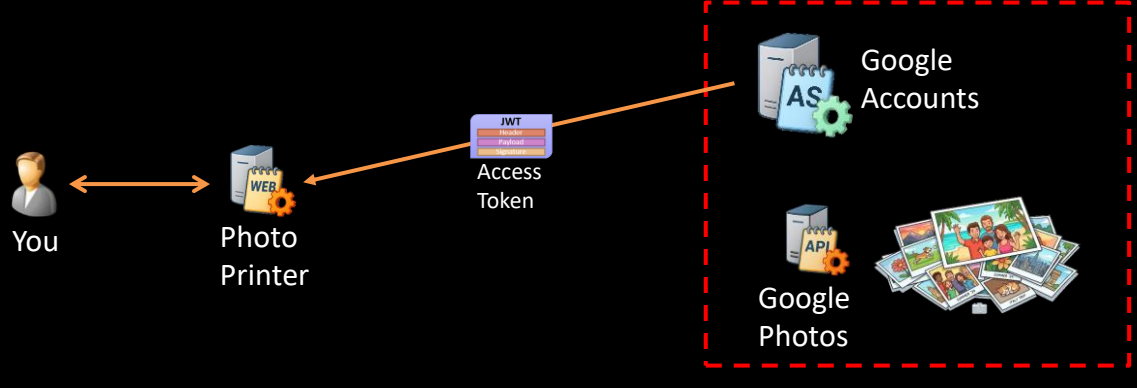
What Happens Next?

https://nestenius.se

13

OpenID Connect

The Tokens Are Returned To the Client



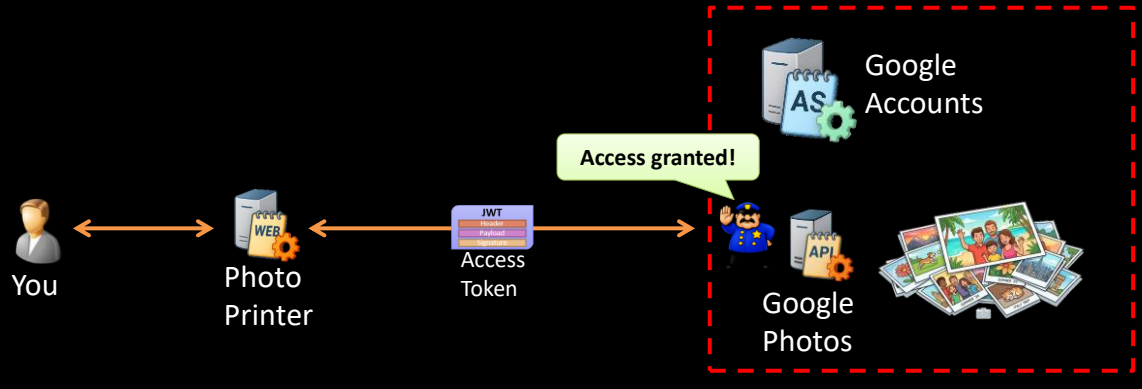
How Does the Photo Printer Use the Token?

https://nestenius.se

14

OpenID Connect

The Photo Printer Uses the Token to Access My Photos



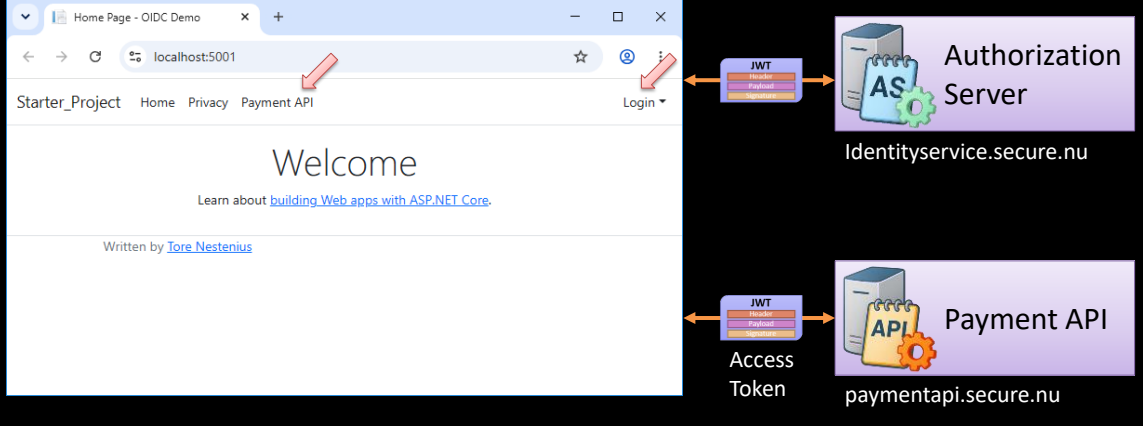
Let us build it in ASP.NET Core

https://nestenius.se

15

Demo #1 – Introducing the Demo Application

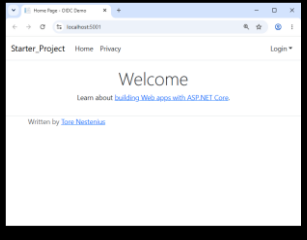
Client Application



https://nestenius.se

16

The Needs of the Client

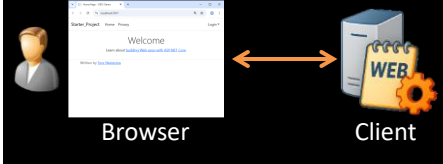


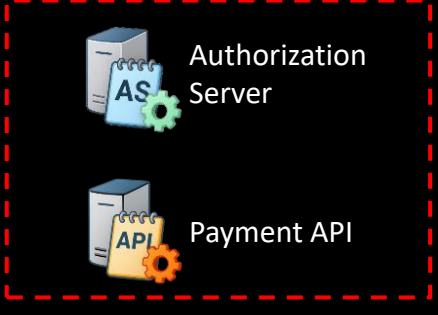
https://nestenius.se

17

The Needs Of The Client

Here is a Typical Setup





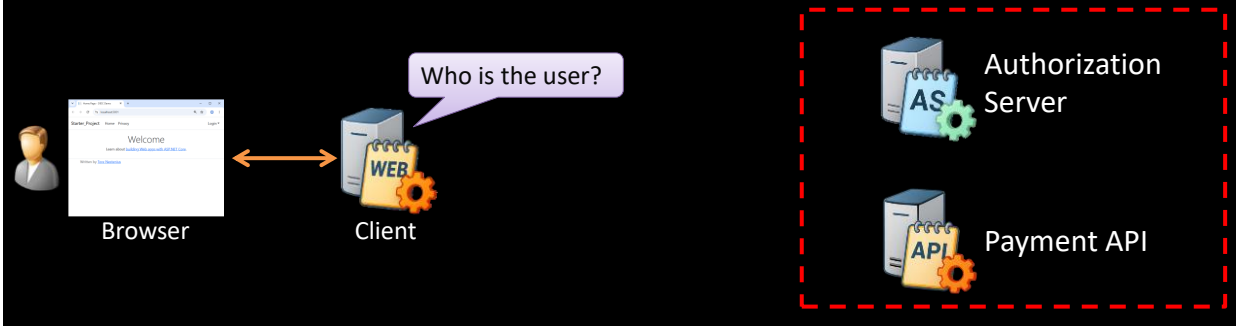
What Does a Client Actually Want?

https://nestenius.se

18

The Needs Of The Client

First, It Needs to Know Who the User Is



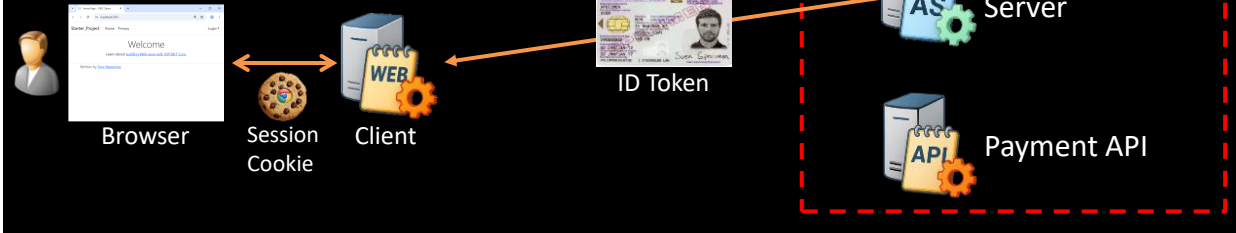
How Does The Client Get This Information?

https://nestenius.se

19

The Needs Of The Client

The Client Gets an ID Token That Identifies the User



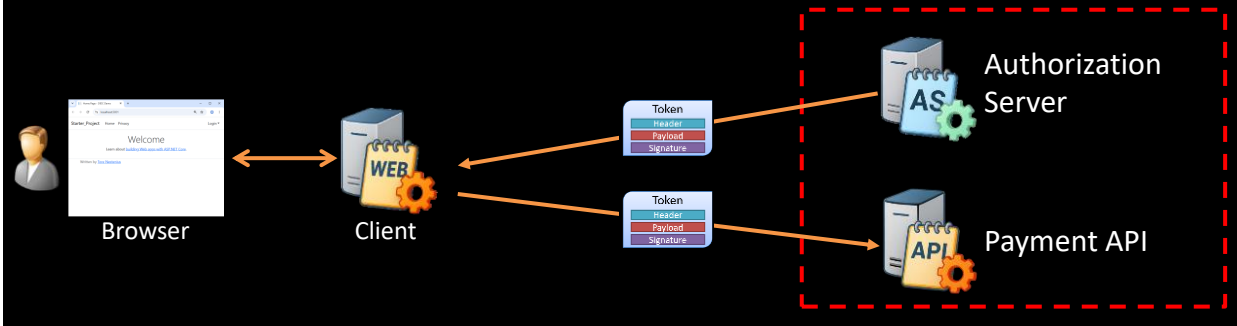
But the Client Also Has Another Need

https://nestenius.se

20

The Needs Of The Client

Second, It Needs to Access the Payment API



The Client Uses the Access Token to Call the API

https://nestenius.se

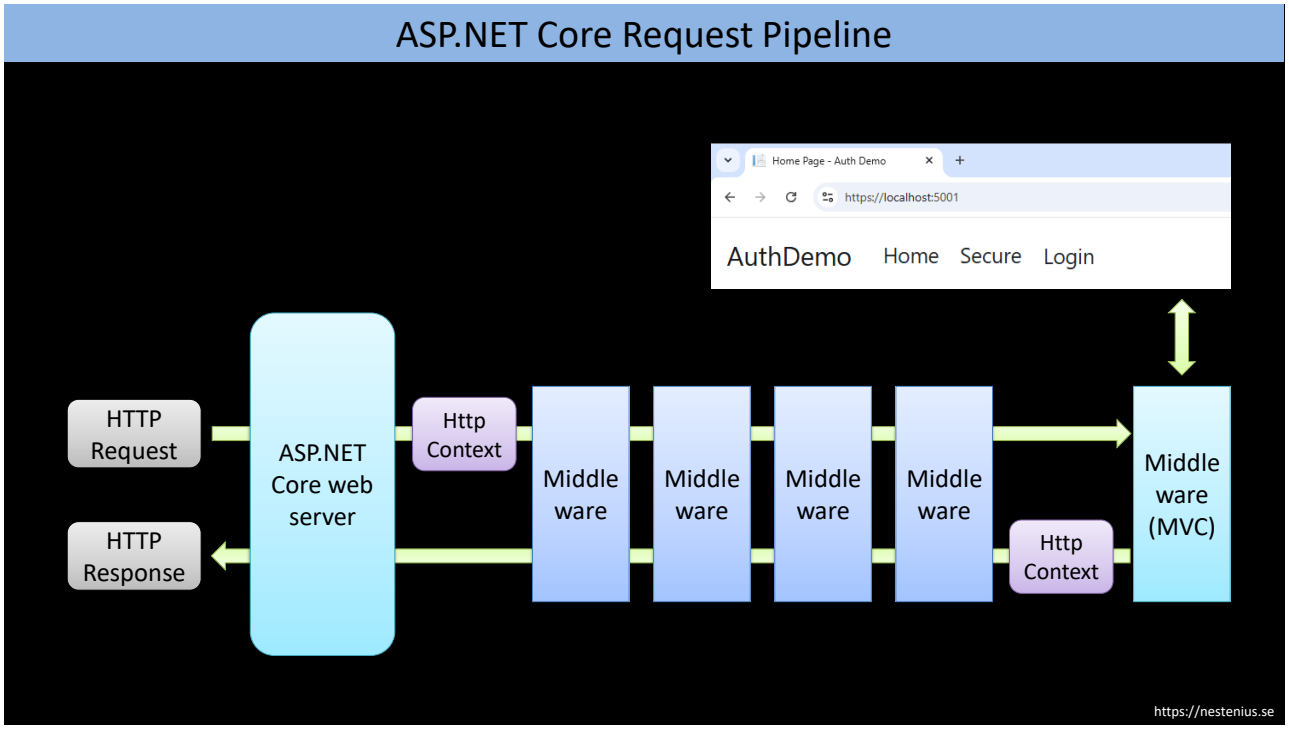
21

ASP.NET Core Request Pipeline

https://nestenius.se

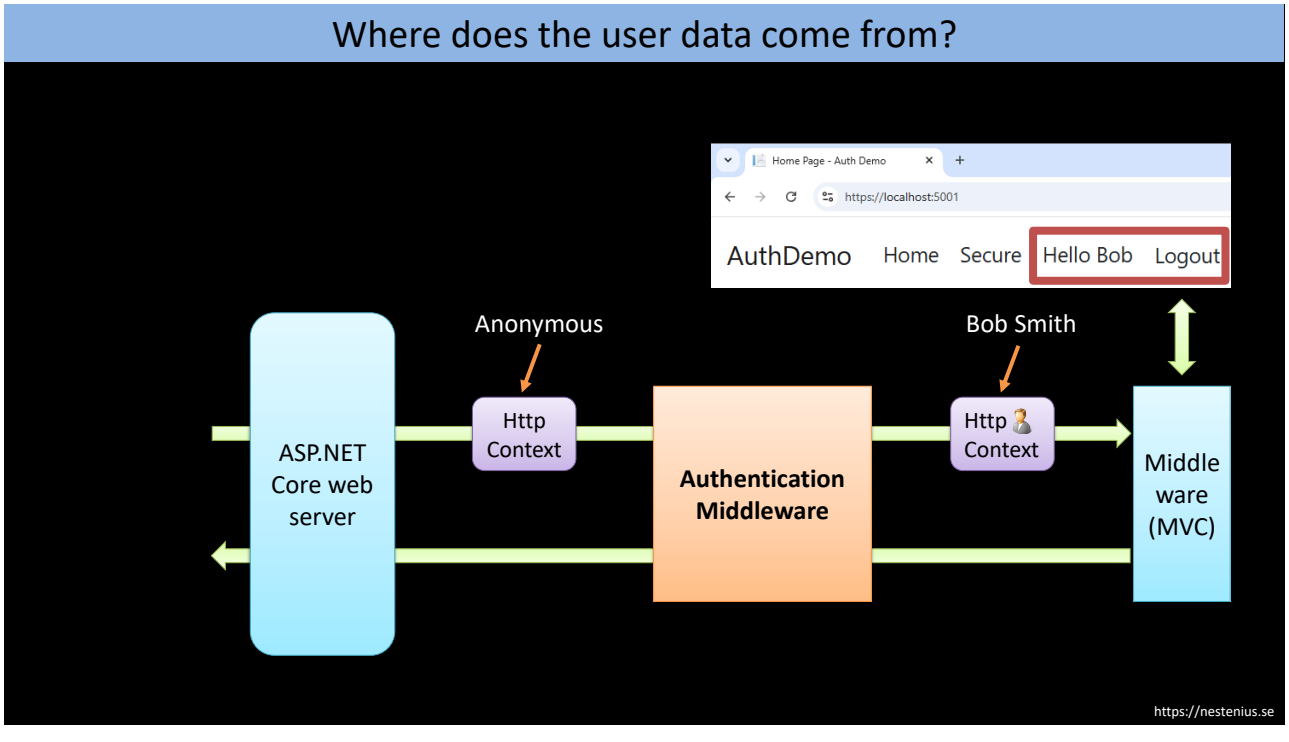
22

ASP.NET Core Request Pipeline



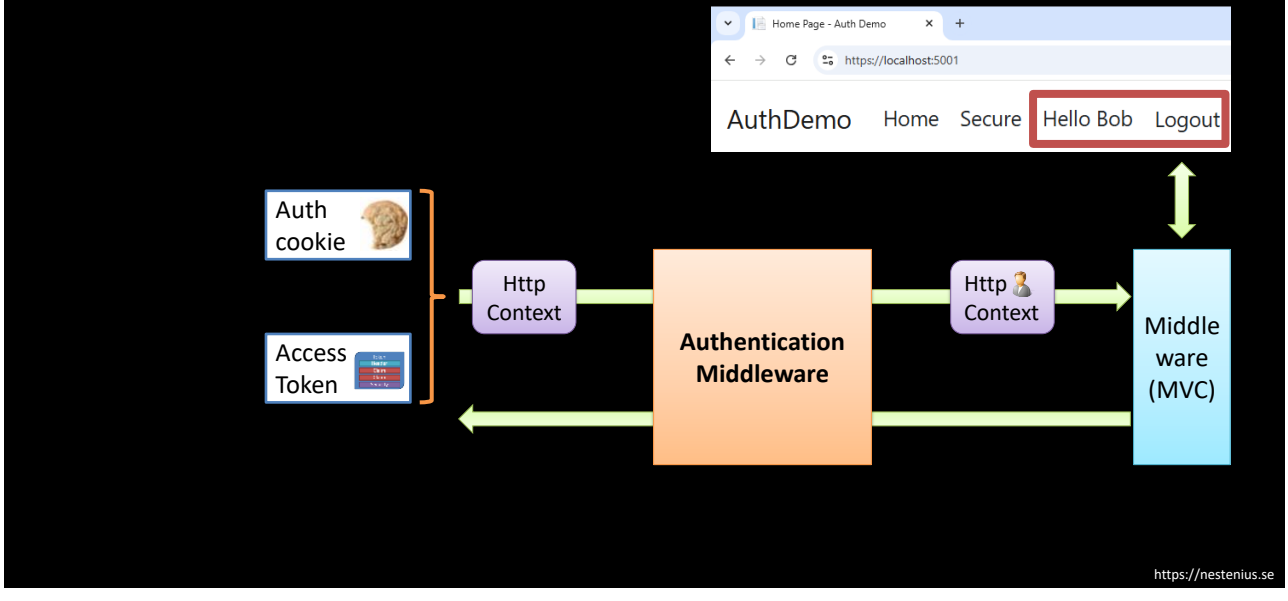
23

Where does the user data come from?



24

Where does the user data come from?



https://nestenius.se

25

Live coding #2 – Adding The Middleware

```
builder.Services.AddAuthentication();

...

app.UseAuthentication();
```

```
public async Task Login()
{
    var prop = new AuthenticationProperties
    {
        RedirectUri = "/"
    };


    // Ask the user to sign in
    await HttpContext.ChallengeAsync("oidc", prop);
}
```

https://nestenius.se

26

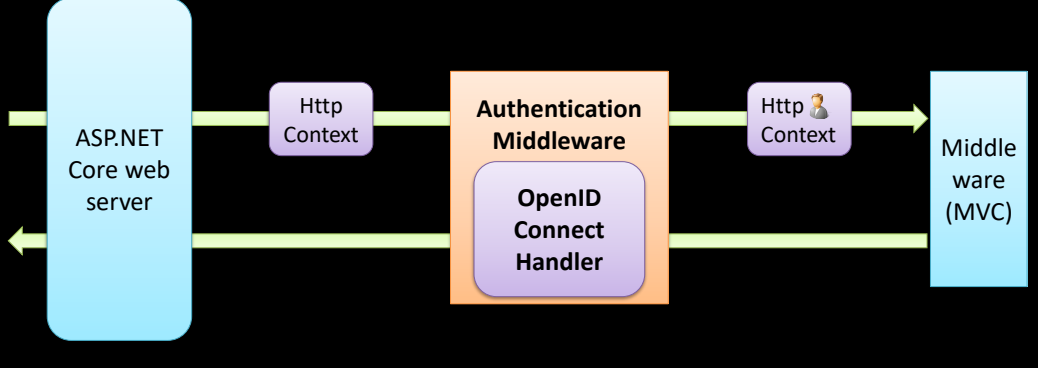
Adding OpenID Connect

https://nestenius.se

27

Adding OpenID Connect

We Start by Adding the OpenID Connect Handler



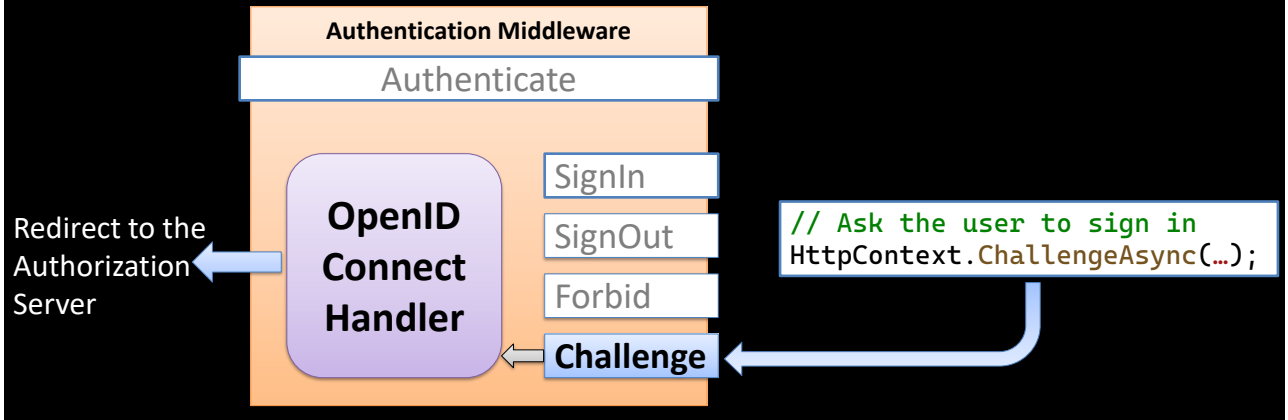
What Is the Handler's Main Purpose?

https://nestenius.se

28

Adding OpenID Connect

It Should Start the Authentication Flow



https://nestenius.se

29

Live coding #3

```
builder.Services.AddAuthentication(o =>
{
    o.DefaultScheme = "cookie";
    o.DefaultChallengeScheme = "oidc";
})
.AddOpenIdConnect("oidc", o =>
{
    o.Authority = "https://identityservice.secure.nu";
    o.ResponseType = "code";

    o.ClientId = "localhost-addoidc-client";
    o.ClientSecret = "mysecret";

    o.Scope.Add("openid");
    o.Scope.Add("email");
    o.Scope.Add("profile");
    o.Scope.Add("payment");

    o.MapInboundClaims = false;
    o.Prompt = "login";
    o.PushedAuthorizationBehavior = PushedAuthorizationBehavior.Disable;
});
```

https://nestenius.se

30

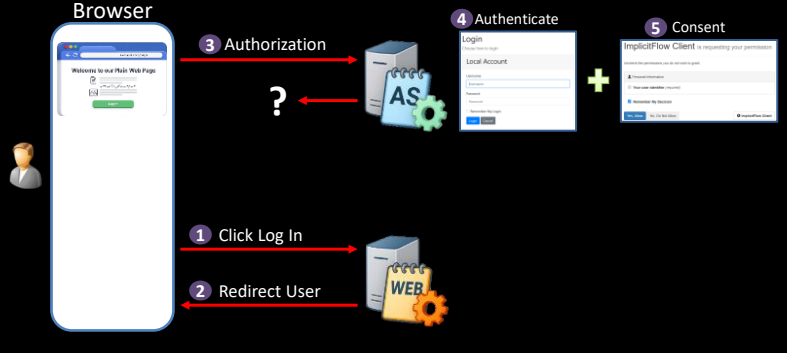
Authorization Code Flow

https://nestenius.se

31

Authorization Code Flow

The User Is Redirected to Sign In and Give Consent



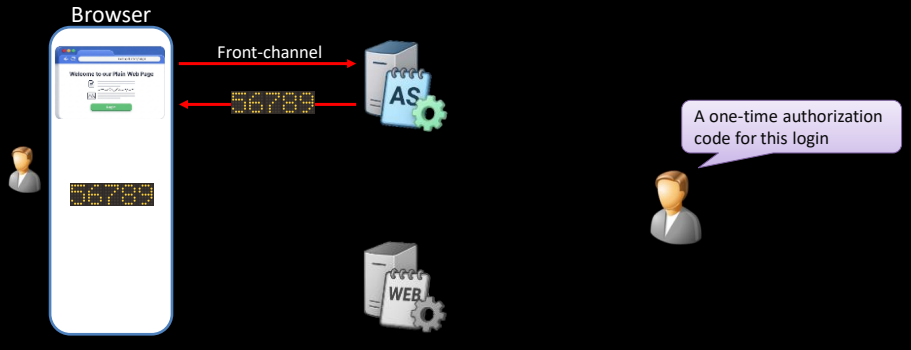
What Is Sent Back After Consent?

https://nestenius.se

32

Authorization Code Flow

An Authorization Code Is Returned



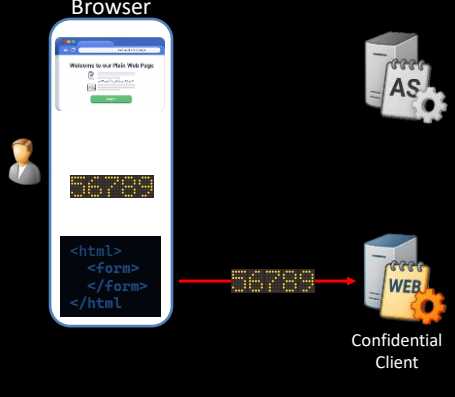
What Happens Next?

https://nestenius.se

33

Authorization Code Flow

The Browser Forwards the Code to the Client



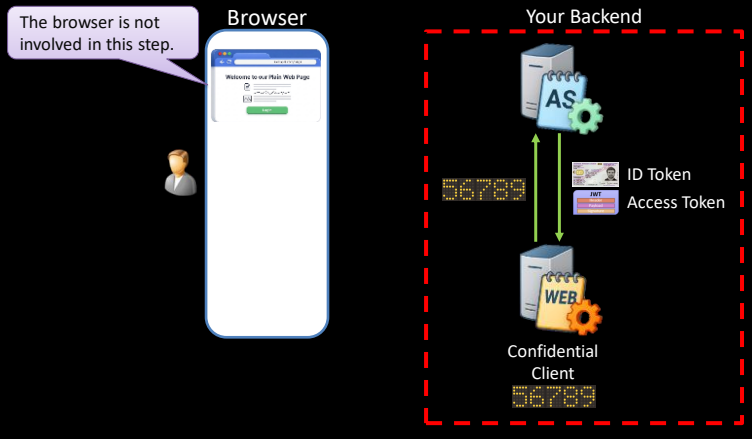
What Does the Client Do With It?

https://nestenius.se

34

Authorization Code Flow

The Client Exchanges The Code For Tokens



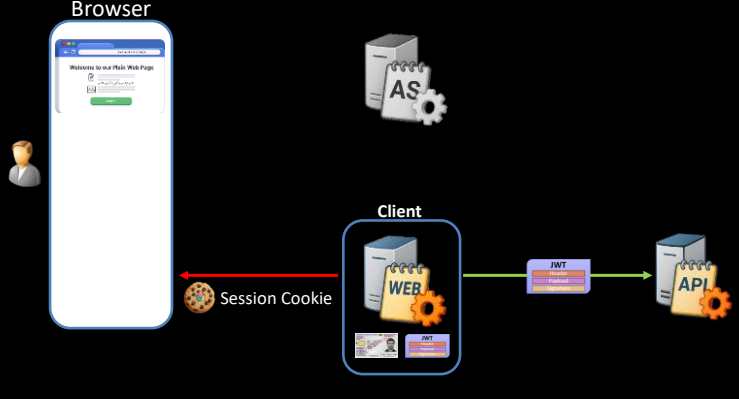
What Happens To The Tokens?

https://nestenius.se

35

Authorization Code Flow

The Client Uses the Tokens



Let us Explore This In Our Demo

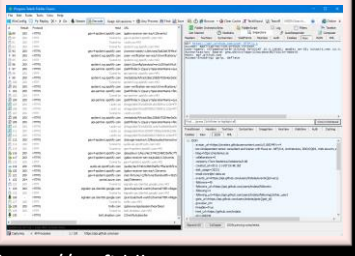
https://nestenius.se

36

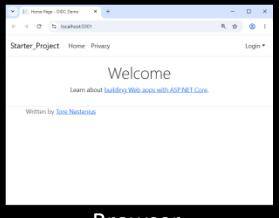
Live coding #4

Authorization Server

Fiddler Classic



https://getfiddler.com



Browser

Client

https://nestenius.se

37

The Missing Piece

An unhandled exception occurred while processing the request.

InvalidOperationException: No sign-in authentication handlers are registered. Did you forget to call AddAuthentication().AddCookie("cookie",...)?

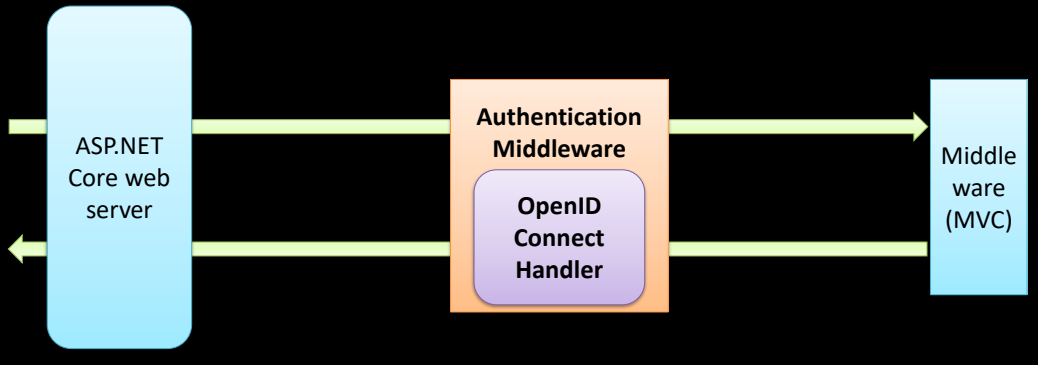
Microsoft.AspNetCore.Authentication.AuthenticationService.SignInAsync(HttpContext context, string scheme, ClaimsPrincipal principal, AuthenticationProperties properties)

https://nestenius.se

38

The Missing Piece

So Far, Our Application Looks like This



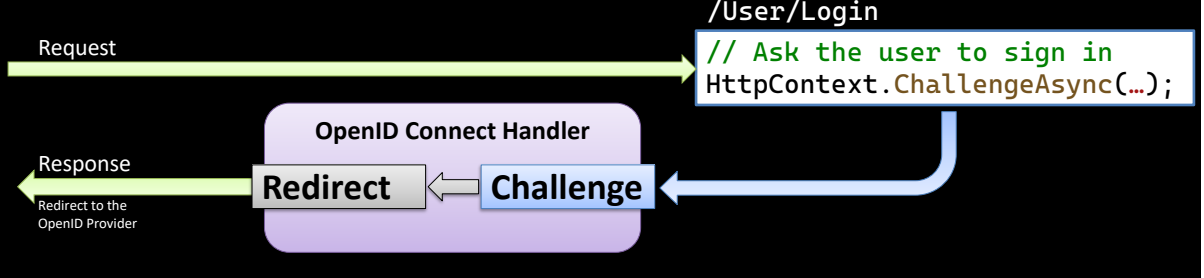
What Has the Handler Done So Far?

https://nestenius.se

39

The Missing Piece

Handle the Authentication Challenge



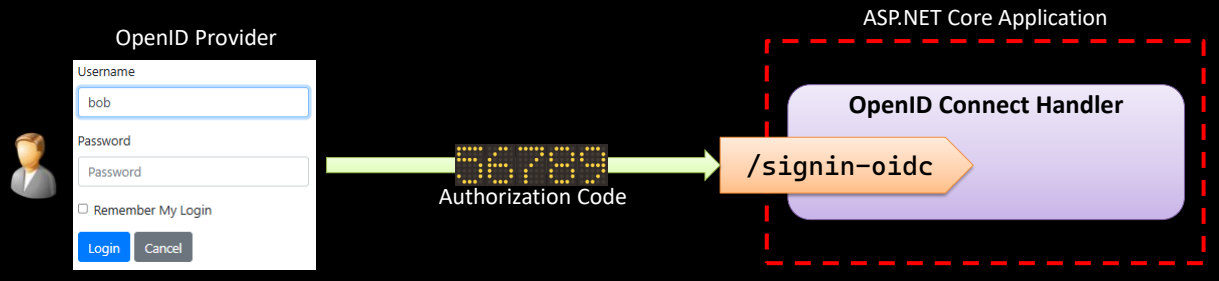
What Else Is It Responsible For?

https://nestenius.se

40

The Missing Piece

Handle the Authentication Callback



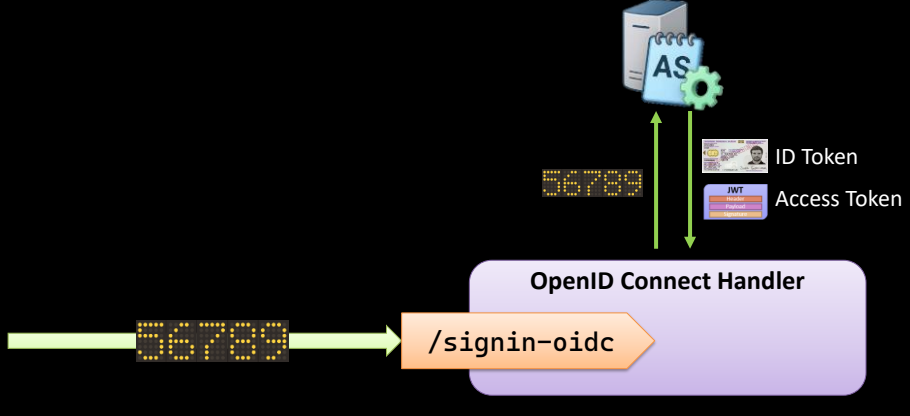
What Happens When It Receives the Code?

https://nestenius.se

41

The Missing Piece

First, It Exchanges the Code for Tokens



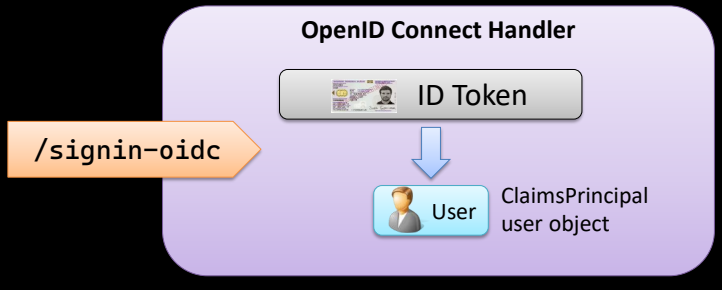
What Happens Next?

https://nestenius.se

42

The Missing Piece

Next, It Creates a ClaimsPrincipal



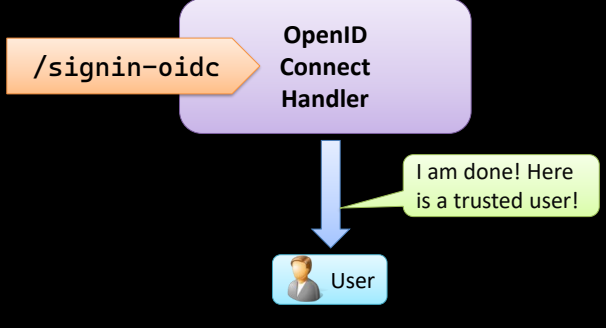
What Does It Do With the User?

https://nestenius.se

43

The Missing Piece

It Needs to Hand Off the User To Someone



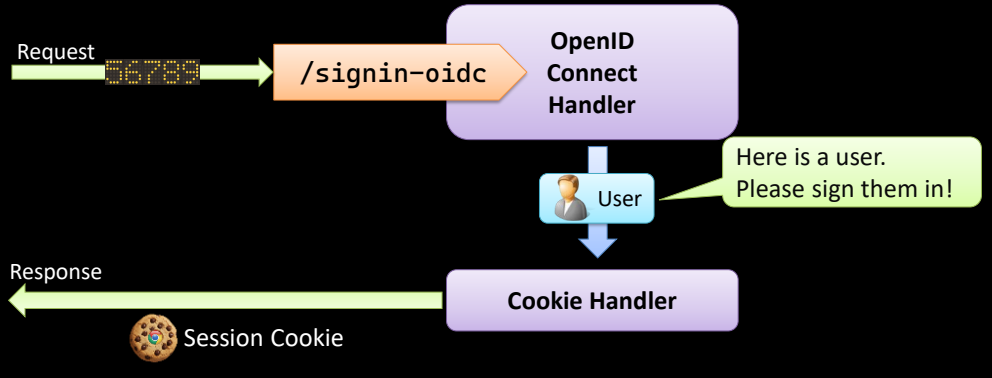
Who Should It Give the User To?

https://nestenius.se

44

The Missing Piece

It Typically Gives It to the Cookie Handler



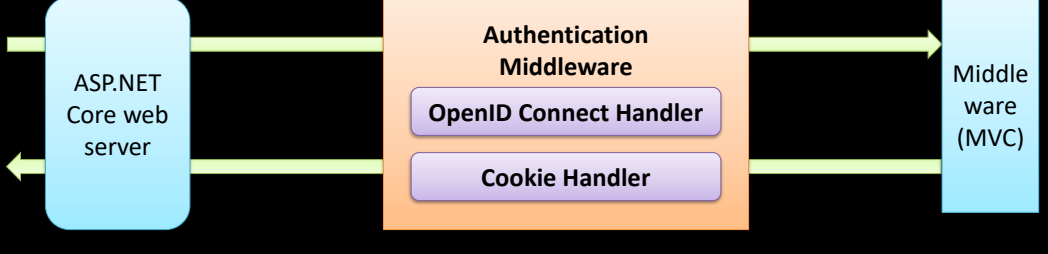
It Will Then Issue a Session Cookie

https://nestenius.se

45

Live coding #5

```
builder.Services.AddAuthentication(o =>
{
    o.DefaultScheme = "cookie";
    o.DefaultChallengeScheme = "oidc";
})
.AddOpenIdConnect("oidc", o =>
{
   ...
}).AddCookie("cookie");
```



https://nestenius.se

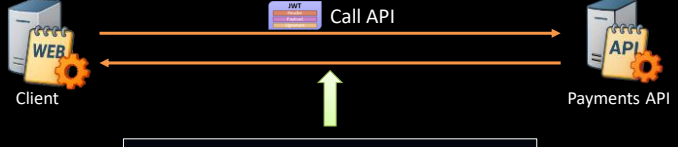
46

Using the Access Token

Using the Access Token

To Call the API We Include the Access Token

```
GET https://payment.secure.nu/payments
Authorization: Bearer eyJhbGciOiJSUzI1NiIsImt...
```




```
HTTP/1.1 200 OK
Content-Type: application/json

{ data }
```

Let Us Implement This!

Live coding #5

```
[HttpPost]
public async Task<IActionResult> GetPayments()
{
    var model = new PaymentApiModel();

    model.AccessToken = await HttpContext.GetTokenAsync("access_token") ?? "";

    var client = new HttpClient();
    client.DefaultRequestHeaders.Authorization =
        new AuthenticationHeaderValue("Bearer", model.AccessToken);

    var response = await client.GetAsync(PaymentsEndpoint);

    model.Result = await FormatResponseAsync(response);

    return View("Index", model);
}
```

https://nestenius.se

49

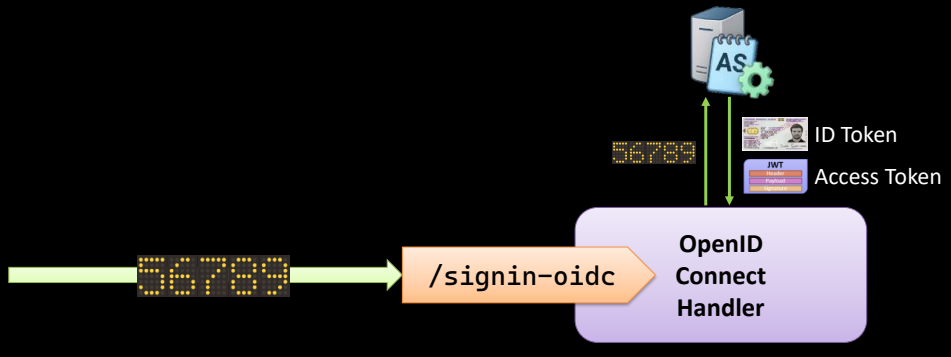
Where Is the Access Token?

https://nestenius.se

50

Where Is the Access Token?

The OpenID Connect Handler Received the Tokens



So, Where Did the Tokens Go?

https://nestenius.se

51

Where Is the Access Token?

By Default, the Tokens Are Not Saved

```
.AddOpenIdConnect("oidc", o =>
{
    ...
});
```

```
.AddOpenIdConnect("oidc", o =>
{
    ...
    o.SaveTokens = true;
});
```

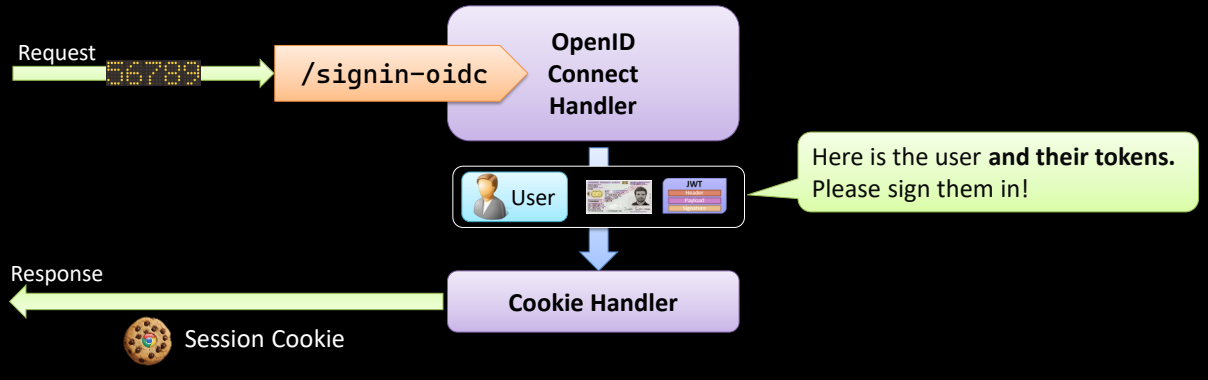
What Happens When We Enable This?

https://nestenius.se

52

Where Is the Access Token?

It Passes the User and Tokens to the Cookie Handler



The Tokens Are Added to the Authentication Session

https://nestenius.se

53

Live coding #6

```
.AddOpenIdConnect("oidc", o =>
{
    ...
    o.SaveTokens = true;
});
```

https://nestenius.se

54

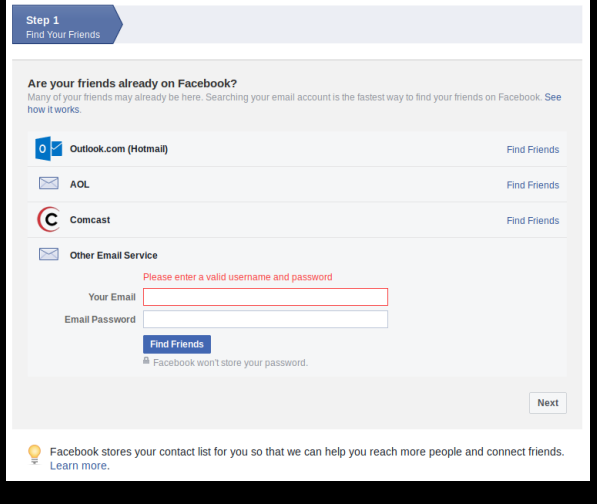
So, What Is OpenID Connect All About?

https://nestenius.se

55

OpenID Connect

The Problem With Sharing Credentials



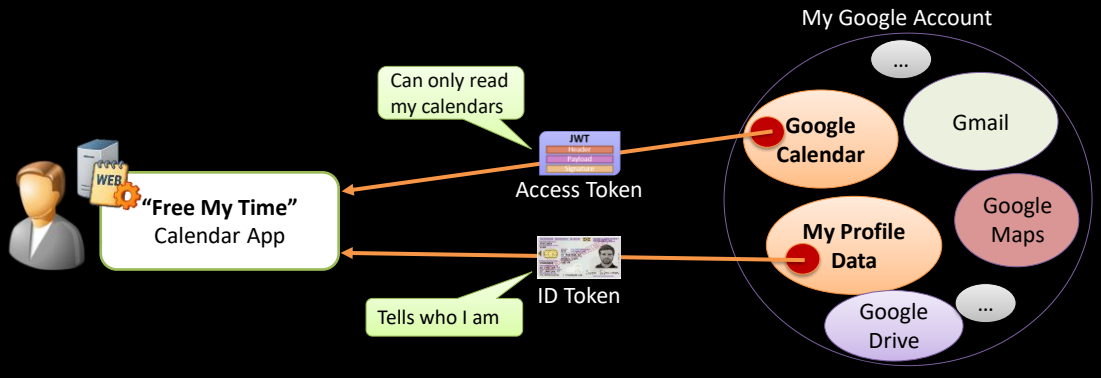
What Do We Want Instead?

https://nestenius.se

56

OpenID Connect

OpenID Connect Gives the Client What It Needs



- I Never Share My Google Password
- I Can Revoke Access at Any Time

https://nestenius.se

57

Questions?

Your next steps:

Scan for slides + Resources

dometrain.com

github.com/tndataab/PublicBlogContent

tn-data.se

58